Waltzz: Extension of a Grammar-Aware WebAssembly Fuzzer with modern WebAssembly Proposals

I. Introduction

Conventional mutational as well as generative fuzzers often struggle when it comes to fuzzing WebAssembly Runtimes. The reasons lies in the fact, that WebAssembly Runtimes enforce a strict validity check before executing a given WebAssembly binary [1]. Primarily the validation mechanism verifies if all instructions are type-checked, meaning for every operation inside of a module it is examined if the previous sequence of instructions generated the stack types consumed by the current instruction on top of the stack.

The growing inclusion of WebAssembly into many different software applications, even beyond the original scope of being an alternative to JavaScript on the web [10], necessitates an extensive framework for testing the correctness of WebAssembly Runtimes to detect dangerous vulnerabilities, before they can be abused by attacker controlled binaries.

The original paper introduces the idea of Stack-Invariant transformations as a way to fuzz WebAssembly Runtimes without being hindered by the Runtimes validation [2]. Instead of randomly flipping bytes, which is the default fuzzing strategy that most state of the art fuzzers including AFL++ use [3], the papers artifact mutates and generates instructions that coincide with the expected Stack-Types. Exemplary an instruction with the stack type $[i32\ i32] \rightarrow [i32]$ can only be replaced by an instruction of the identical stack type.

A thorough evaluation of different metrics comparing the performance of Waltzz to other popular fuzzers, came to the conclusion that apart from reaching its goal of creating strictly valid test cases, the artifact had 12.4% more code coverage and discovered 1.38× more bugs [2].

Despite the improvements of the overall fuzzing capabilities, Waltzz still showcases some limitations in its current form. The custom fuzzing process focuses on WebAssembly MVP instructions, while the same paper states that about 40% of recently discovered bugs come from newer less furbished WebAssembly concepts [2]. Furthermore the custom Waltzz mutator displays significantly lower executions per seconds compared to the integrated AFL++ mutators, deeming it useful to optimize the custom fuzzing process. Beyond that, other shortcomings like the non existent mitigation for the creation of infinite loops during the generation process, while still remaining were not considered during the development of this extension. To address these problems I extended Waltzz incrementally with 3 non MVP WebAssembly proposals: Threads(Atomics) [4], Multi-Memory [5] and Relaxed-SIMD [6]. Further I have identified hot code paths, which take up most of the execution time and are optimizable without changing the core workflow of Waltzz, and have improved them through a O(1) hash instruction lookup and stack allocated child lookups. Concretely, this work:

- Extends the mutators group tables and instruction parsing/generating with atomic and relaxed-SIMD instructions following the same equivalent stack-type exchange principals as the original.
- Adds a multi-memory allocator, which randomly creates additional memories for a module through the use of newly programmed Binaryen API calls, as well as expanding the splicing mechanism to correctly handle splicing two modules together without destroying the referenced memory context.
- Changes the heap allocation of AST node children extraction to a faster stack allocation.
- Replaces the slow linear search of instruction groups with a faster O(1) hash table approach.
- Showcases improvements in terms of execution speeds and possible instruction sequences that can be created by Waltzz.

II. Background

Waltzz was developed as a custom mutator that sits on top of AFL++. Therefore it can use the underlying fuzzing architecture of AFL++ like scheduling and coverage feedback, while itself focusing on specifically fuzzing WebAssembly binaries and reaching the goal of solely generating valid WebAssembly modules.

To achieve that goal the authors designed a custom stack-invariant fuzzing mechanism. The main idea behind a stack-invariant transformation, is that every instruction inherently defines its stack types, describing what type of values the instruction pops from the stack and what type of values the instruction pushes back onto the stack: $[t_1^*] \rightarrow [t_2^*]$. For instance a simple 32 bit integer subtraction instruction $i32.sub$ needs to consume two $i32$ values from the stack and ends up pushing the result as another $i32$ value onto the stack, therefore defining its stack type as: $[i32\ i32] \rightarrow [i32]$. Now to keep the validity of an already valid module, the stack-invariant transformation only replaces code segments with other segments when both stack-types are equivalent, since

that does not change the stack types verified by the validation mechanism.

Instead of randomly changing raw bytes in the binaries, the mutator parses these binaries through the Binaryen library [8] into an intermediate representation in the form of an abstract syntax tree, storing the module, the instructions, and so on into nodes. Also grouping every instruction definition with equivalent stack types into the same instruction groups allows the mutator to pick an instruction from the module, check its instruction group, search through the corresponding group for other valid candidates and replace them without destroying the present validity of the module. Waltzz then makes use of this through six different mutators: call target mutator, interesting value mutator, branch target mutator, operator mutator, recursive mutator and splicing mutator. While each mutator focuses on fuzzing other parts of the module, they all at their core analyze current stack types and only change them with instruction sequences of matching stack types.

However at the point of publication, Waltzz was developed to only fuzz MVP WebAssembly as well as bulk memory and SIMD instructions [2]. This hinders the ability to maximize the code coverage of the to be tested runtime since without active incorporation of new proposals Waltzz does not have the ability to create these code paths.

III. Contribution

Following from the MVP focused nature of Waltzz, the main point of extension lies in integrating support for new non MVP features of WebAssembly. Using the already established structure of Waltzz, extending the mutation and generation processes was straight forward for many instructions, resulting in the extension of the mutator files as well as the addition of new Binaryen bridging files, consisting of more than 500 lines of C/C++ code. The main problem arising when trying to include multi-memory support as creating multiple memories naively inside of a single module breaks some of the fuzzing logic.

A. Threads (Atomics)

The first feature I added were the relevant parts of the Threads proposal. While defining the multi threading like workflow of multiple so called agents independently executing modules inside of an agent cluster, my extension focuses only on the added atomic instructions and shared memory support [4]. The proposal defines 67 new instructions designed to interact with memory either shared or unshared. All instructions can be categorized into the groups: Load/Store, Read-Modify-Write, Compare Exchange, Wait/Notify and a single fence operation. Apart from these new instructions the proposal also adds the ability for a memory to be declared either shared, in proposal context meaning that all agents in an agent cluster can see the changes on this memory, however in the Waltzz implementation context without actual thread behavior being just another state a memory can be in or unshared, which just describes the normal state of the linear memory. Important to consider is that while most atomic instructions work on shared and unshared memory, the wait instructions trap if called on unshared memory, which would decrease the effectiveness of the fuzzing process.

The starting point of the actual development of this feature is the same as the rest. First the feature flag must be enabled in `wasm_mutators.c` to include the Binaryen thread capabilities.

Then the inclusion of the new atomic instructions into the fuzzing process is achieved by extending two parts of the custom mutator. During simple operator mutation Waltzz searches linearly through an array of instruction groups until the correct stack type group is found. Extending this with the new instructions consists of filling these instructions into their corresponding stack type group using the additionally added macro `FILL_INSTR_TYPE_OPD_MEM_ATOMIC`, which also sets the correct values of the newly added fields inside of the instruction descriptor `InstrTy`.

Other than using the initialized instruction groups to fuzz the module, Waltzz makes use of multiple helper functions from `helper_func.c`, that parse and set data for and of a given instruction, which are then further used inside the generator and mutators. Extending this part was similar to the instruction groups, since the functions just needed new if cases for checking atomic instruction identifications and extracting or setting the correct information. One thing that needs extra attention is the trapping potential of misaligned memory accesses. Unlike normal memory instructions, atomic memory instructions trap when they are misaligned [4], needing a well formed alignment setting compared to the randomized setting of normal instructions.

The last addition from the Threads proposal was the ability to create shared linear memory. While the entire memory creation workflow of Waltzz was changed working on multi-memory support, the simple extension idea for shared memory is still the same. The memory setter function `wasm_set_memory` now `wasm_set_memory_count` was simply expanded to include one additional parameter `bool guarantee_shared`, which then forces a shared memory creation based on the boolean value of the parameter. Now when a memory is created during the fuzzing process using this function, the value of `guarantee_shared` is randomized, since most instructions can work on either shared or unshared memory, the exception being when dealing with the atomic wait operator, which causes a forced shared memory creation.

B. Multi-Memory

This proposal adds the functionality to create more than one memory inside of a module [5]. Before this proposal WebAssembly modules could only contain up to one defined memory, causing every data entry to be stored in the same location but with this extension multiple independent memories can be defined for different usages inside of a module. While it does not add any new instructions to the WebAssembly language itself, it uses the already defined fields from memory

related instructions to make them able to construct or target different memories.

Implementing multi-memory support into Waltzz seems simple at first, but runs into issues, especially with the used Binaryen version 117 [8]. When it comes to fuzzing multiple memories with Waltzz it would be useful to gather information about the memory state of the module and to set multiple memories without interfering with the other memories. Binaryen however does not expose suitable functions through the C-API to get information about any other memory than the default one, get the memory name based on a given data segment or index or reliably set new memories without deleting previously defined ones.

Instead of directly writing functions within the Binaryen library to get the needed functionality, as I tried at first but reverted from that idea due to concerns about future proofing the extension for when updates to the Binaryen library may come into consideration, I wrote a bridging file `binaryen_bridge.cpp`. This bridging file uses the underlying structure and data stored in Binaryen objects to achieve the wanted memory functionality. While in total adding 9 new functions for multi-memory access, the most notable ones are `WaltzzGetNumMemories()`, which returns the number of memories defined in the given module, `WaltzzGetMemoryNameByIndex()`, returning the memory name at the wanted index in the memories array and `WaltzzAddMemory()`, which unlike the native Binaryen counterpart that deletes every other memory from the stored memories array, simply adds the new memory onto the array, without interacting with the other memories.

Having now the ability to work with multi-memory inside of Waltzz, the actual fuzzing implementation can follow. The main idea is that every time when a memory instruction is mutated or generated, it randomly picks the memory name it targets, through `wasm_pick_memory_name()` and `wasm_pick_two_memory_names()` for the memory copy instructions. The functions work by randomly choosing an index between 0 and the defined `GENERATED_MEMORY_MAX` value, ensuring that a memory at the chosen index exists and the memory fits the needed criteria and finally returning the memory name the index points to. To ensure the existence of memories up to that index, the function `wasm_set_memory_count()` is called passing the randomly generated wanted amount of memories. Memories are then generated up to that amount, if they do not currently exist, while also checking if shared memories or passive data segments are present, when they are needed by the caller.

Executing the extension with these additions however, caused the fuzzer to crash after some time. The error always pointed to a faulty module being created and looking deeper stemmed from instructions targeting memories, that do not exist. After further inspections of the problem I came to the conclusion that splicing modules while having multiple memories breaks the naive baseline approach. In a single memory environment the validity of splicing fragments from a donor module into a recipient module just comes down to the availability of a memory in general, since every instruction just targets the same default memory. With multiple memories however a fragment could reference a memory index/name that does not exist in the recipient module or does not support the needed sharedness criteria for example. This mismatch of targeted and available memories causes the crashes, because Binaryen cannot build a module when an expression exists that targets an undefined memory name. To eliminate this issue I needed to implement a memory splicing fix up.

The correct splicing behavior can be achieved through the use of the two new helper functions `wasm_find_splice_recipient()` and `wasm_remap_spliced_memory_references()`. The latter function iterates over every child starting from a root expression and scans, if a memory access is present, what memory name the current expression targets. For every memory name encountered for the first time, a remap takes place to a memory existing inside of the recipient module through `wasm_find_splice_recipient()`. The remap follows the structure of first trying an identical memory name from the recipient with matching attributes, if that does not work any declared recipient memory, which is not already used by another remap and as a final fallback a fresh clone of the donor memory. This order is used to keep the number of created memories to a minimum, preventing the module from being inflated due to repeated splicing during the fuzzing campaign. Remapping the targeted memories ensures that every memory instruction spliced into the recipient module is accepted by the Binaryen module builder.

C. Relaxed-SIMD

The Extension of Relaxed-SIMD support is very similar to the extension of atomic instructions [6]. In total the proposal adds 20 instructions already known through normal SIMD, the difference lying only in the non-deterministic execution of those instructions.

When it comes to implementing Relaxed-SIMD support into Waltzz, it is sufficient, apart from again activating the feature in `wasm_mutators.c`, to extend the corresponding instruction initialization groups inside of `init_func.c` with the new Relaxed-SIMD instructions.

An extension of the helper functions is not needed, since Waltzz works with the already supported overarching instruction types, which can either be `unary, binary` or `simdTernary` for overall SIMD instructions. So Waltzz can handle the mutation and generation of Relaxed-SIMD instructions, without any specific parsing cases.

D. Optimizations

When fuzzing with Waltzz the fuzzer first uses native AFL++ mutators before switching to the custom Waltzz mutator. Comparing these two groups of mutators in terms of executions per second, the custom WebAssembly mutator runs significantly slower than the AFL++ mutators. While the main goal of my extension is still the addition of new proposals

to increase the amount of possible WebAssembly modules to be generated, quickening the execution speed of the custom mutator is still beneficial especially when considering the evaluation process.

During a single iteration of a mutation taking place, Waltzz parses the WebAssembly binary through Binaryen into an AST. The mutations and generations then take place based on the nodes inside of that tree, before transforming the mutated AST back into an executable WebAssembly module. Parsing WebAssembly files is a heavy task, which takes up most of the time during one iteration, being the main cause of the slow execution speed. Optimizing this part of the fuzzing process however is not a feasible extension for this project, since that would mean a complete redesign and new implementation of the core working mechanism behind Waltzz.

Apart from that, analyzing parts of the code showed other optimization candidates, with simpler possible extensions designs.

One function, which is called a lot during fuzzing is the helper function `wasm_get_children_expr()`, which returns the children of the passed parent node. To return the list of children, the function allocates memory on the heap, based on the amount of children the current node has, using `calloc()`. Even though most parent nodes have a ceiling of 4 children, the function still uses slow heap allocation, wasting execution time, when a stack allocation would be more efficient. For my slight optimization the caller now passes a buffer of fixed size 4 to the function, which then fills the array with the children. Since the function also returns the number of children there is no danger of reading the array past the wanted index. For the few cases, where the number of children is variable the function simply still uses the `calloc()` method.

The second optimization aims to improve the speed at which the corresponding instruction group of a given instruction is found. In the base version this is done through a linear search of every instruction inside of each instruction group until the correct group is found in the function `wasm_find_instr_grps()`. A linear search is already slow for large arrays and scales with every added instruction, having an $O(total_instructions)$ time complexity. To improve the search for an instruction group, I extended the `wasm_init_instr_grps()` function to build a hash table with open addressing linear probing, where the already calculated hash value of an instruction is the key and the value is the corresponding instruction group. With the use of that hash table `wasm_find_instr_grps()`, now instead of linearly searching for the matching group, has an average lookup complexity of $O(1)$.

IV. Evaluation

To verify if the extension of these proposals and slight optimizations of the execution actually improve the fuzzing capabilities, it is important to compare the base version of Waltzz with my written extension.

The main goal of this extension was increasing the amount of possible WebAssembly instruction sequences the fuzzer can produce, to uncover new code paths of the to be tested WebAssembly runtime and therefore increase edge coverage and possibly uncover new bugs.

The less important result of my extension through the small optimizations, should cause the custom mutator to run slightly faster during the fuzzing process compared to the base version.

The expected result of the comparison should show increased code coverage from my extension compared to the base version, unique instructions and instruction sequences appearing in the generated WebAssembly binaries of my extension, which are impossible to be constructed by the base version and a higher average execution speed.

A. Setup

The runtime experiments were conducted on an AWS EC2 Graviton instance running 64-bit Ubuntu 24.04 LTS Arm-architecture with 16vCPU and 32 GB of main memory.

Both the base version of Waltzz and my extension were tested against the same non-web embedded WebAssembly runtime wasmtime (version 22.0.0) [9]. In total 8 instances of each the base version and my extension ran for a period of 24 hours.

Given that the fuzzer AFL++, which serves as the basis of Waltzz is a mutational fuzzer a seed corpus is needed, that consists of initial valid WebAssembly binaries to be mutated by the fuzzing algorithm. These initial files were inspired by the official WebAssembly test suite [7] and turned into a small seed corpus of 12 valid WebAssembly modules.

B. Comparison of Edge Coverage

After a period of 24 hours that each test instance ran I had a total of 16 fuzzing statistics, 8 for the base version and 8 for the extended version. While most of these instances ran as expected, 2 instances of the base version crashed before starting their 24 hour runs without saving any fuzzing statistics, due to complications with free CPU cores and 2 runs of the extended version displayed a significant execution speed difference of just 115.64 execs/sec and 119.09 execs/sec respectively, where the other extension runs all had speeds of at least 2000 execs/sec. Given the speed discrepancy of these 2 runs in combination with the 2 failed runs from the base version, I decided to use the correct functioning 6 runs for each version.

The results of averaging the fuzzing statistics can be seen in “Table I”, for both the base version and the extension.

During this evaluation my extension on average had 43% faster execution speeds for the overall fuzzing process. While this could also be attributed to the common variance of AFL++ fuzzing speeds, that significant of an increase at least suggests that the hash table lookup and stack allocations improvements had a positive impact on the execution speed. Applying a one-sided Mann-Whitney U test on the given fuzzing data resulted in a p-value of 0.0022, which is less than the conventional significance threshold of 0.05, making that difference statistically significant. The results of the Mann-Whitney U test

should still be interpreted with caution, since the sample size of just 6 campaigns is really small.

In terms of edges uncovered by the fuzzers, the extended version had statistically insignificant less coverage at -0.7%. This difference can be explained by the natural fuzzing variance, since AFL++ and therefore Waltzz uncover edges due to random mutations, so it is a typical result that the coverage differs slightly from test to test. To get an actual verifiable increase of edge coverage due to the proposal extensions, the tests would have to run until the base version of Waltzz could no longer find valuable edges and the extension hit the edges, which are impossible for the base version to uncover.

TABLE I
COMPARISON OF FUZZING STATISTICS AFTER 24 HOURS

Metric	***Base***	***Extension***	Δ	p[a]
Execs per second	1620.27	2317.125	+43%	0.0022
Unique Edges found	60526.5	60083	-0.7%	0.093

[a]Mann-Whitney U.

C. Comparison of created Instruction Types

To get an actual understanding of what merit my extension offers, we have to look at what kind of modules Waltzz now creates and in what way this differs to the creation process of the base version.

Analyzing every WebAssembly module generated by the extended Version of Waltzz, totals in 111857 files from the 6 considered runs. As shown in "Table II" the new proposals are present in many of these generated files. This shows that my extension has a considerable influence over the fuzzing process, especially when it comes to the generation of multiple memories, which are present in more than 80% of fuzzer generated modules. While at the current evaluation from "Table I" it can not be said concretely that my extension improves the range of coverage, the amount of unique instruction sequences impossible by base Waltzz to be created, implies that the extended version increases the range of possible edges to be uncovered.

For the matter of low frequency of Relaxed-SIMD instructions seen in "Table II", this is an expected result, since the conditions for mutating and generating such sequences are much rarer to be encountered during a fuzzing campaign.

TABLE II
FEATURE FREQUENCY IN EXTENSION CORPUS (111857 FILES IN TOTAL)

Feature	***Files***	***% of corpus***
Atomics	42477	37.97%
Multi-Memories	91166	81.5%
Relaxed-SIMD	5375	4.8%

V. CONCLUSION

This project extended the base version of a grammar-aware custom AFL++ mutator called Waltzz. With the idea of stack-invariant transformations and the given code structure of the program, the extension includes the generation and mutation of atomic instructions, multiple memories and relaxed-SIMD instructions. This is mostly achieved by implementing support for the given instructions inside stack-typed instruction groups and helper functions. Multi-memory support required more sophisticated extensions, especially with custom Binaryen C-API functions and a completely new WebAssembly memory generation workflow. Apart from the proposal extension, the fuzzer was also optimized through prioritized stack allocation instead of heap allocation in an often called helper function and a switch to an $O(1)$ hash table instruction group lookup compared to the $O(total_instructions)$ linear search.

During the evaluation process, the difference in execution speeds could be measured at 43% more executions per second compared to the base version. Even after considering the normal fluctuation of fuzzing speeds, such a difference still points to an optimized process due to my extensions.

While the extensive fuzzing campaign showed a clear speed improvement, it did not display a significant difference in edge coverage. This however does not conclude that the extension has no impact on said coverage, but just that the runs were still not long enough to uncover the expected difference. What can be proven is the creation of new instructions, unique to my extensions. Atomics as well as multiple memories can be found in a significant amount of generated files, 37% and 81% respectively. The creation of those instruction sequences is impossible for the base version and shows a decisive improvement of the range of modules that can be created by the custom mutator.

Even with these extensions, there is still room for improvement in the future. For one, especially when considering the inconclusive edge coverage evaluation, longer test runs could lead to more concrete results in the metrics that matter. During these improved test sets, using Waltzz on multiple different runtimes, will most likely also give a broader and more detailed analysis of the changes the extension provides. Following the same ideas that were already the inspiration for this extension, reasonable future work on this project could contain the extension of even more WebAssembly proposals. These can include ones already suggested by the original paper like garbage collection, or any one of the phase 4 or phase 5 WebAssembly features, that fit in current workflow of Waltzz. Looking at the already improved execution speed, still leaves room for even better performance, so future work could also go in the direction of optimizing the hottest code paths further, maybe even changing the core workflow to switch from the slow parsing caused by Binaryen to a more efficient method.

REFERENCES

[1] W3C. The WebAssembly specification validation algorithm. https://webassembly.github.io/spec/core/appendix/algorithm.html

[2] Zhang, L., Zhao, B., Xu, J., Liu, P., Xie, Q., Tian, Y., ... & Ji, S. (2025). Waltzz:WebAssembly Runtime Fuzzing with Stack-Invariant Transformation. In 34th USENIX Security Symposium (USENIX Security 25) (pp. 6159-6178).

[3] Fioraldi, A., Maier, D., Eißfeldt, H., & Heuse, M. (2020). AFL++: Combining incremental steps of fuzzing research. In 14th USENIX workshop on offensive technologies (WOOT 20).

[4] The WebAssembly Threading(Atomics) proposal. https://github.com/WebAssembly/threads/blob/main-legacy/proposals/threads/Overview.md

[5] The WebAssembly Multiple Memories proposal. https://github.com/WebAssembly/multi-memory/blob/main/proposals/multi-memory/Overview.md

[6] The WebAssembly Relaxed-SIMD proposal. https://github.com/WebAssembly/relaxed-simd/tree/main/proposals/relaxed-simd

[7] W3C. The official test suite for core WebAssembly. https://github.com/WebAssembly/spec/tree/main/test/core.

[8] WebAssembly compiler and toolchain library Binaryen. https://github.com/WebAssembly/binaryen. Version 117

[9] Rust based WebAssembly runtime wasmtime. https://github.com/bytecodealliance/wasmtime.

[10] WebAssembly non-web embeddings. https://webassembly.org/docs/non-web/